



Data Flow 49: Where `$derived` Belongs in the Product Tree

Lecture 49 teaches `$derived` as the rune for keeping a local variable continuously synced with an incoming prop, plus the Svelte 5.25 twist that lets you reassign that derived value for optimistic UI (see Lecture 49 for how `$derived` works). This file does not re-teach the mechanism; it shows *where* that sync belongs in ElectroShop, and why one tier beats the two on either side of it. The running example is a product's wishlist count: a number that arrives on the `product` prop, must stay current as the grid re-renders, and must jump instantly when the user clicks the heart before the server confirms.

A) The Technical Svelte Information

The `$derived` rune declares a value that recalculates whenever any reactive state read inside its expression changes. For prop sync, `let wishlistCount = $derived(product.wishlistCount)` reads `product.wishlistCount` at setup and then re-reads it every time the parent hands down a new `product`, keeping the child in continuous sync without the one-shot deafness of `$state` (the trap Lecture 49 opens with). Because the derived expression is a pure function of its inputs, Svelte also permits a temporary reassignment (`wishlistCount += 1`) that holds until the next recalculation washes it away, the mechanism behind optimistic UI.

B) Large Scale Data Flow Context

ElectroShopApp (Holds global user session) → *ShopLayout* → *MainContent* → *ProductGrid* (Holds raw product array, iterates) → ***ProductCard*** (Where the prop sync lives) → *ProductActions* → *WishlistButton* (Renders the count, fires the click)

C) Nuanced Small Scale Data Flow

ProductGrid (passes one raw `product` object as prop per card) → *ProductCard* (receives `let { product } = $props()`) → **`$derived(product.wishlistCount)`** (one-line sync; re-derives on every prop change) → *WishlistButton* (receives `wishlistCount`, renders the heart, optimistically overrides on click)

Why this placement

1. **Why not derive in `ProductGrid` ?** The grid's job is to iterate. If it owned each card's derived count, any single product update would force it to walk the whole array and re-derive every card. Putting `$derived` inside `ProductCard` means each card owns exactly one derivation, scoped to its own prop, and the grid stays a dumb iterator.
2. **Why not derive in `WishlistButton` ?** The button is a presentational leaf: render a heart, show a number, fire a click. Lifting the sync into it couples display to the data-plumbing contract with the parent and forces the button to know what a `product` prop is. The derived value should arrive at the button already computed.
3. **Why `$derived` and not `$state` here?** The count's source of truth is the parent's `product` prop, not the card. `$state(product.wishlistCount)` would snapshot the value once at card creation and go deaf to every later prop change, the precise trap Lecture 49 exists to prevent. `$derived` keeps the card reactive to the grid above it.
4. **Why the sync and the override live in the same tier.** The optimistic override (`wishlistCount += 1`) only works if the derived value and the click handler share scope. `ProductCard` owns both: it declares `$derived(product.wishlistCount)` and passes an `onclick` down to `WishlistButton` . When the click fires, the card bumps its derived value locally; when the server confirms and the grid hands down a fresh `product` , the derivation recalculates and the override is

quietly replaced by the new source of truth.

● ● ● ProductCard.svelte

```
1  <script>
2    let { product, addToWishlist } = $props();
3
4
5    let wishlistCount = $derived(product.wishlistCount);
6
7    async function handleWishlist() {
8
9      wishlistCount += 1;
10     try {
11       await addToWishlist(product.id);
12     } catch {
13       wishlistCount -= 1;
14     }
15   }
16 </script>
17
18 <WishlistButton count={wishlistCount} onclick={handleWishlist} />
```

Continuous sync with the parent's prop, never a one-shot snapshot

1. optimistic override: bump the count **LOCALLY** before the server answers

2. tell the server to **SAVE** the click

3. rollback: **REVERT** the override if the request failed